ProLecture

Бенчмарк проактивности, упирающейся в рассуждение

Модель смотрит лекцию в реальном времени и сама решает,
когда ответить и **что** ответить.

Материал — лекции 3Blue1Brown. Статус: пилот, 10 сэмплов, конвейер разметки собран.

Ниша

проактивность там, где ответ требует вывода, а не считывания

Работа	Что меряет	Чего не хватает
OVO-Bench	backward / real-time / forward	MCQ, нет аудио, перцепция
ProactiveVideoQA	open-ended, метрика PAUC	простые вопросы «сообщи, когда X»
OmniPro	6 способностей $\times$ 9 подзадач	84% аудио, но vision-only не отстают

- В лекции почти **нет перцептивной сложности**: чистый рендер, крупные формулы, чёткая речь.
- Зато есть логическая структура: посылка $\rightarrow$ вывод $\rightarrow$ следствие.
- Значит, ответить вовремя можно, только **поняв содержание** — а не распознав объект.

Почему 3Blue1Brown

уникальный актив: у нас есть исходный код картинки

Для **140** видео известны одновременно ссылка на YouTube и существующий путь к manim-исходнику сцены.

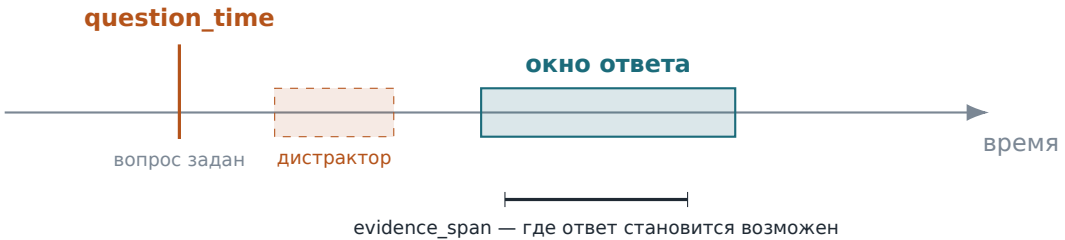
Ни у одного из существующих бенчмарков исходников видеоряда нет.

- точная ground-truth того, что на экране (строки Tex с LaTeX);
- семантические границы: имена сцен, порядок анимаций;
- **ре-рендер с внедрённой ошибкой** — контент, которого гарантированно не было в pretraining.

Связь «урок ↔ ролик ↔ код» существует только во frontmatter сайта: в самом репозитории `3b1b/videos` нет ни одной ссылки на YouTube.

Анатомия сэмпла

вопрос — в одну секунду, ответ — в окне, доказательство — отдельно



- Ось `temporal_rel` (`pre` / `at` / `post`) **выводится валидатором** из положения доказательства, а не ставится руками.
- `distractor_windows` — моменты, где ответ выглядит уместным, но преждевременен.
- Подтверждение эталона живёт отдельно, в `provenance`: оно может лежать позже окна и даже вне ролика.

Три оси таксономии

каждый сэмпл — точка в кубе (акт, отношение, уровень рассуждения)

A. Тип акта

`alert` — сигнал
`monitor` — состояние
`ground` — локализация
`narrate` — описание
`predict` — предсказание
`verify` — верификация
`counterfact` — контрфактика

B. Отношение к триггеру

`pre` — доказательство после вопроса: **умение ждать**

`at` — здесь и сейчас

`post` — ответ по памяти

Один вопрос в двух режимах даёт контролируемую пару.

C. Уровень рассуждения

`L1` перцепция — считывается

`L2` интеграция — связать два момента

`L3` вывод — провести рассуждение, которого в видео нет

Критерий L3: blind-бейзлайн не решает.

Ось C — то, чего нет у OmniPro: там модели не различаются именно по ней.

Принципы, из которых вырос дизайн

каждый пункт закрывает провал, на котором погорела одна из разобранных работ

- **Метки ставятся замером, а не декларацией.** `audio_dependency` выводится из ответов зондов модальностей; мнение разметчика живёт в отдельном поле и на выводы не влияет.
- **Генератор $\neq$ судья $\neq$ участник.** Три разные модели.
- **Открытые ответы с рубрикой, без MCQ.** Перекошенный ключ ответов ломает бенчмарк тихо.
- **Обязательные бейзлайны-пустышки:** blind, text-only, vision-only, frozen-frame, «отвечать сразу», «отвечать в конце», «молчать».
- **Никаких квот** вида «ровно N сэмплов на задачу»: сколько прошло валидацию, столько и есть.
- **Контаминация измеряется, а не отрицается:** пилот на видео 2026 года, контрольное видео 2017 года — чтобы разрыв blind-бейзлайна увидеть числом.

Метрика

PAUC плюс то, что PAUC не наказывает

PAUC. На каждом окне строится ломаная «время $\rightarrow$ качество накопленных ответов», качество $s \in \{0, 1, 2\}$ ставит LLM-судья по рубрике.

Стартовая точка $(t_{start}, 0.5)$: молчание лучше уверенного неверного ответа.

Следствие, которое отчитываем явно: стратегия «молчать всегда» получает **0.25** от максимума. Это пол шкалы, а не ноль.

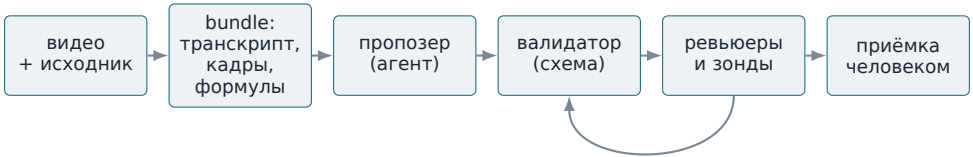
Обязательно рядом:

- **False-Alarm Rate** — ответы вне окон
- **Trap rate** — попадания в дистракторы, отдельно от FAR
- **Latency** — от t_{start}, распределением
- **Silence rate**, **Trigger Recall**
- **Responses-per-sample** — иначе спам-стратегия невидима

Агрегация по сэмплам, а не по окнам: иначе итог — заложник одного самого простого сэмпла с девятью триггерами.

Конвейер

от ролика до сэмпла, прошедшего ревью



- **Полу-слепой ревьюер** видит ролик до конца окна, но не знает его начала — и называет момент, когда ответ стал возможен. Совпало ⇒ границы честные.
- **Зонды модальностей** (text-only, vision-only, frozen-frame, полный) дают метку `audio_dependency` замером.
- **Валидатор** — роль тестов: непересекающиеся окна, ссылки только назад, дистрактор не совпадает с триггером, уникальность `id`.

Инструменты, которых не было

два места, где готовое не подошло

Свой детектор смены сцен

Готовые детекторы сравнивают кадр с предыдущим — а типовой переход manim это плавный переезд за 2–4 секунды, где соседние кадры почти не отличаются.

Считаем **накопленный дрейф** от последнего снятого кадра и снимаем, когда анимация улеглась.

Один порог на домен: **13.5** / **12.9** / **10.1** кадра на минуту на роликах 2026 и 2017 годов.

На контрольном участке: **18** состояний из 18 против **10** у равномерной сетки.

Плотное описание видеоряда

Документ вроде субтитров, только вместо реплик — что происходит на экране. Разметчик читает текст вместо просмотра десятков кадров.

Два кадра в секунду, окна по 5 секунд с перекрытием, строгий JSON по схеме (guided decoding на стороне сервера).

Пилот: **3878** кадров, **485** фрагментов, **20** минут на двух A100, **1939** секунд из 1939 покрыты событиями.

Субтитры описывающей модели не показывают: она описывает только кадры. Иначе описание нельзя было бы отдать зонду vision-only вместо кадров.

Где мы сейчас

Сделано

- конвейер данных: маппинг → скачивание → bundle;
- схема сэмпла и валидатор семантики;
- пилот: **10** сэмплов, **24** триггера и **12** дистракторов, ошибок валидации ноль;
- промпты пропозера, трёх ревьюеров и четырёх зондов;
- детектор сцен и описание видеоряда.

Дальше

- прогнать бейзлайны-пустышки и увидеть, что они проваливаются;
- замерить разрыв blind на контрольном видео 2017 года;
- harness потоковой выдачи и LLM-судья по рубрике;
- расширение разметки: доля L3 должна вырасти;
- опционально — ре-рендер сцен с внедрённой ошибкой.

Открытые допущения записаны отдельно и в Limitations, а не заметены: утечка объяснения дистрактора, высокая сложность пилота, малый объём.